

Web Development Front-End



Trabalho Final Teórico

Aluno: Marco Antonio Monteiro Pedro

Sumário

Parte Teórica – Galeria de Personagens dos Simpsons.....	3
Introdução Geral.....	3
1.O que é uma API e como funciona o consumo de dados em tempo real.....	3
1.1 Conceito de API	3
1.2. Como funciona a requisição cliente-servidor	3
1.3. Consumo em tempo real	4
1.4. Funções envolvidas na requisição	4
1.5. Exemplo prático do fluxo	5
1.6. Conclusão técnica	5
2 Conceito de DOM Manipulation	5
2.1. O que é o DOM	5
2.2. Manipulação dinâmica com JavaScript	5
2.3. Exemplo prático do nosso código.....	6
3. Integração com o fluxo da API	7
.....	7
3.1. Usuário busca um personagem	7
3.2. JavaScript recebe os dados da API (JSON)	7
3.3. Para cada personagem, chama createHeroCard(hero).....	8
3.4. Montagem dinâmica do HTML.....	8
3.5. Inserção no container.....	9
3.6. Resultado para o usuário	9
3.7. Conclusão.....	9
4. Importância da DOM Manipulation	10
4.1 Conclusão	10
4.2 Funções básicas utilizadas (detalhado)	10
5. Integração entre as funções	12
5.1 Tratamento de erros e robustez da aplicação.....	12
6. Motivo da escolha da API dos Simpsons (detalhado)	13
6.1. Relevância cultural.....	13
6.2. Facilidade técnica	13
6.3. Estrutura dos dados	13
6.4. Aplicação prática no projeto	14
6.5. Justificativa final.....	14
7. Regras de acesso e documentação da API	14
Conclusão Geral.....	15

Parte Teórica – Galeria de Personagens dos Simpsons

Introdução Geral

O presente trabalho tem como objetivo demonstrar, de forma prática e teórica, o funcionamento da integração entre aplicações web e APIs públicas, utilizando como exemplo a criação de uma galeria interativa de personagens da série *Os Simpsons*. A proposta busca evidenciar como conceitos fundamentais de programação — como consumo de dados em tempo real, manipulação do DOM e utilização de funções básicas em JavaScript — podem ser aplicados para transformar informações brutas em interfaces dinâmicas e amigáveis ao usuário.

Ao longo do desenvolvimento, são explorados os principais elementos que compõem essa integração: desde a requisição cliente-servidor e o tratamento da resposta em formato JSON, até a construção dinâmica de elementos HTML que exibem os dados de forma visual. A escolha da API dos Simpsons se justifica tanto pela relevância cultural da série, ícone dos anos 90 e 2000, quanto pela facilidade técnica de acesso, já que se trata de uma API pública e gratuita, sem necessidade de autenticação.

Dessa forma, o trabalho não apenas atende aos requisitos acadêmicos propostos, mas também evidencia a importância da prática de programação voltada para aplicações reais, mostrando como a combinação entre teoria e código resulta em soluções robustas, escaláveis e interativas.

1. O que é uma API e como funciona o consumo de dados em tempo real

1.1 Conceito de API

Uma **API (Application Programming Interface)** é um conjunto de regras que permite que diferentes sistemas conversem entre si. No caso da web, uma API expõe dados em formato estruturado (normalmente **JSON**) que podem ser consumidos por aplicações front-end. Isso significa que o navegador não precisa ter os dados armazenados localmente: ele pede ao servidor e recebe a resposta em tempo real.

1.2. Como funciona a requisição cliente-servidor

O fluxo básico é:

1. O **navegador (cliente)** dispara uma requisição HTTP para o **servidor da API**.
2. O servidor processa a requisição e retorna uma resposta em formato JSON.
3. O cliente interpreta esse JSON e transforma em elementos visuais (cards, tabelas, etc.).

No código, isso acontece assim:

javascript

```
const response = await fetch(BASE_URL);
```

```
const data = await response.json();
```

- `fetch(BASE_URL)` → envia uma requisição HTTP GET para o servidor da API dos Simpsons.
- `response.json()` → converte o corpo da resposta (texto bruto) em objeto JavaScript manipulável.

1.3. Consumo em tempo real

O consumo em tempo real significa que **cada vez que o usuário digita um nome e clica em “Buscar”**, o navegador faz uma nova requisição ao servidor. Se a API atualizar os dados (por exemplo, adicionar novos personagens), o usuário verá essa atualização imediatamente, sem precisar recarregar manualmente a página.

Exemplo no código:

javascript

```
button.addEventListener("click", async () => {
```

```
  const name = input.value.trim();
```

```
  const heroes = await searchHero(name);
```

```
  // Aqui os dados já vêm atualizados direto do servidor
```

```
});
```

1.4. Funções envolvidas na requisição

- `searchHero(name)`: encapsula a lógica da requisição.
 - Faz o `fetch` para a API.
 - Converte a resposta em JSON.
 - Filtra os personagens pelo nome digitado.

javascript

```
return data.docs.filter((hero) =>
```

```
  hero.Nombre.toLowerCase().includes(name.toLowerCase())
```

```
);
```

- **Servidor da API**: recebe a requisição e retorna um JSON com todos os personagens.
- **Front-End (nosso código)**: interpreta esse JSON e cria os elementos HTML dinamicamente.

1.5. Exemplo prático do fluxo

1. Usuário digita “Homer” no campo.
2. O navegador dispara `fetch("https://apisimpsons.fly.dev/api/personajes?limit=1000")`.
3. O servidor responde com um JSON contendo todos os personagens.
4. O código filtra apenas os que têm Nombre contendo “homer”.
5. Para cada resultado, `createHeroCard(hero)` monta um card com nome, imagem, ocupação e estado.
6. O card é inserido no DOM com `appendChild()`.

1.6. Conclusão técnica

- A API funciona como **ponte de dados** entre servidor e cliente.
- O consumo em tempo real garante que os dados exibidos sejam sempre os mais atuais.
- O `fetch` é a função central que dispara a requisição HTTP.
- O `response.json()` transforma a resposta em objeto manipulável.
- O front-end usa manipulação de DOM para transformar dados em interface visual.

2 Conceito de DOM Manipulation

2.1. O que é o DOM

O **DOM (Document Object Model)** é a representação em memória da estrutura de um documento HTML. Ele transforma cada elemento da página (tags como `<div>`, `<h1>`, `<p>`, etc.) em **nós** e **objetos** que podem ser acessados e manipulados via JavaScript. Assim, o DOM funciona como uma ponte entre o **código HTML estático** e a **interatividade dinâmica** que o usuário vê no navegador.

2.2. Manipulação dinâmica com JavaScript

Com JavaScript, podemos:

- **Criar elementos:** `document.createElement("div")` cria um novo `<div>`.
- **Alterar atributos:** `element.setAttribute("class", "card")` adiciona uma classe.
- **Inserir conteúdo:** `element.textContent = "Homer Simpson"` coloca texto dentro de um elemento.
- **Adicionar ao DOM:** `container.appendChild(element)` insere o novo elemento na página.

Essas operações permitem que a página seja atualizada sem recarregar, criando uma experiência interativa.

2.3. Exemplo prático do nosso código

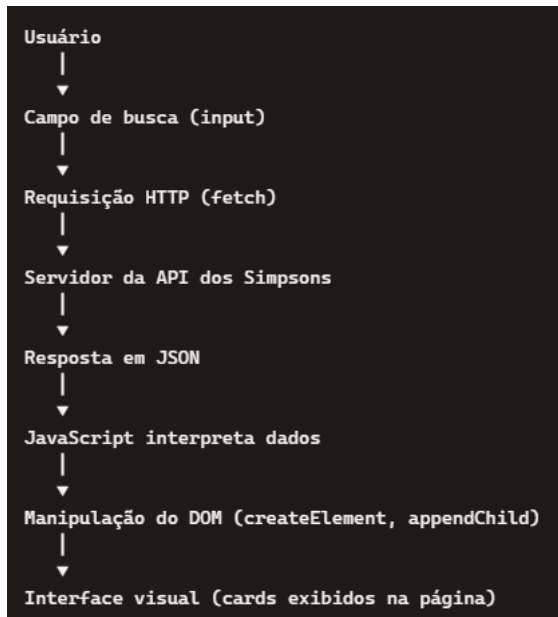
No projeto dos Simpsons, usamos a função `createHeroCard(hero)` para montar os cards dinamicamente:

javascript

```
function createHeroCard(hero) {  
  const card = document.createElement("div");  
  card.className = "card";  
  
  const name = document.createElement("h2");  
  name.textContent = hero.Nombre;  
  
  const img = document.createElement("img");  
  img.src = hero.Imagen;  
  
  const ocupacion = document.createElement("p");  
  ocupacion.textContent = `Ocupação: ${hero.Ocupacion}`;  
  
  const estado = document.createElement("p");  
  estado.textContent = `Estado: ${hero.Estado}`;  
  
  card.appendChild(name);  
  card.appendChild(img);  
  card.appendChild(ocupacion);  
  card.appendChild(estado);  
  
  return card;  
}
```

- `document.createElement()` → cria novos elementos HTML.
- `textContent` → insere texto dentro dos elementos.
- `appendChild()` → adiciona cada elemento ao card.
- No final, o card é retornado e inserido no container da página.

3. Integração com o fluxo da API



3.1. Usuário busca um personagem

Quando o usuário digita um nome no campo de busca e clica no botão, o **JavaScript** captura esse valor e dispara a função principal:

javascript

```
button.addEventListener("click", async () => {
  const name = input.value.trim();
  const heroes = await searchHero(name);
  ...
});
```

- O evento de clique é registrado com `addEventListener`.
- O valor do input é lido e passado para a função `searchHero(name)`.

3.2. JavaScript recebe os dados da API (JSON)

A função `searchHero` faz a requisição HTTP usando `fetch`:

javascript

```
async function searchHero(name) {
  const response = await fetch(BASE_URL);
  const data = await response.json();
```

```

return data.docs.filter((hero) =>
  hero.Nombre.toLowerCase().includes(name.toLowerCase())
);
}

```

- `fetch(BASE_URL)` → envia requisição GET para o servidor da API dos Simpsons.
- `response.json()` → converte a resposta em objeto JavaScript.
- `data.docs` → contém o array de personagens.

3.3. Para cada personagem, chama `createHeroCard(hero)`

Se a busca retorna resultados, o código percorre o array com `forEach`:

javascript

```

heroes.forEach((hero) => {
  const card = createHeroCard(hero);
  container.appendChild(card);
});

```

- Cada objeto `hero` contém atributos (`Nombre`, `Imagen`, `Ocupacion`, `Estado`).
- A função `createHeroCard(hero)` monta dinamicamente o HTML correspondente.

3.4. Montagem dinâmica do HTML

Dentro de `createHeroCard(hero)`:

javascript

```

function createHeroCard(hero) {
  const card = document.createElement("div");
  card.className = "card";

  const name = document.createElement("h2");
  name.textContent = hero.Nombre;

  const img = document.createElement("img");
  img.src = hero.Imagen;

  const ocupacion = document.createElement("p");
  ocupacion.textContent = `Ocupação: ${hero.Ocupacion}`;

```

```
const estado = document.createElement("p");
estado.textContent = `Estado: ${hero.Estado}`;
```

```
card.appendChild(name);
card.appendChild(img);
card.appendChild(ocupacion);
card.appendChild(estado);
```

```
return card;
```

```
}
```

- createElement() cria novos elementos HTML.
- textContent insere texto.
- appendChild() adiciona cada elemento ao card.
- O card final é retornado para ser exibido.

3.5. Inserção no container

O card é inserido no container principal da página:

javascript

```
container.appendChild(card);
```

- O container (#cards-container) recebe os novos elementos.
- O navegador renderiza os cards imediatamente.

3.6. Resultado para o usuário

O usuário vê os **cards renderizados na tela sem recarregar a página**. Isso acontece porque:

- O **fetch** busca dados em tempo real.
- O **JavaScript** manipula o DOM dinamicamente.
- O **HTML** recebe os elementos criados via código.

3.7. Conclusão

Essa integração mostra como o front-end consome dados da API e transforma JSON em interface visual:

1. Input do usuário → dispara evento.
2. fetch → requisita dados ao servidor.

3. JSON → convertido em objetos JavaScript.
4. createHeroCard → monta HTML dinamicamente.
5. appendChild → insere no container.
6. Resultado → cards exibidos em tempo real.

4. Importância da DOM Manipulation

- **Flexibilidade:** permite criar interfaces dinâmicas sem depender de HTML fixo.
- **Interatividade:** o usuário interage com a página e vê mudanças imediatas.
- **Escalabilidade:** podemos consumir qualquer API e transformar os dados em elementos visuais.
- **Separação de responsabilidades:** o HTML fornece a estrutura básica, enquanto o JavaScript gera o conteúdo dinâmico.

4.1 Conclusão

A manipulação do DOM é o coração da parte prática do projeto. É ela que transforma os dados vindos da API dos Simpsons em **cards visuais interativos**. Sem o DOM, teríamos apenas dados brutos em JSON; com ele, conseguimos criar uma interface amigável e dinâmica para o usuário.

4.2 Funções básicas utilizadas (detalhado)

4.2.1. *fetch()*

- **O que é:** função nativa do JavaScript usada para enviar requisições HTTP (GET, POST, etc.) e receber respostas de servidores.
- **Como funciona:** retorna uma *Promise*, que representa o resultado futuro da requisição.
- **No nosso código:**

```
javascript
```

```
const response = await fetch(BASE_URL);
```

- Aqui o navegador envia uma requisição GET para o servidor da API dos Simpsons.
- O servidor responde com dados brutos em formato JSON.

4.2.2. *then()* / *await*

- **O que são:** mecanismos para lidar com operações assíncronas.
 - *then()* é usado em Promises para encadear ações após a resposta.
 - *await* é usado dentro de funções *async* para “pausar” a execução até que a Promise seja resolvida.
- **No nosso código:**

javascript

```
const data = await response.json();
```

- O await garante que o código só continue depois que a resposta da API for convertida em JSON.
- Sem await, o código poderia tentar usar os dados antes de estarem prontos.

4.2.3. *createElement()*

- **O que é:** função do DOM usada para criar novos elementos HTML dinamicamente.
- **No nosso código:**

javascript

```
const card = document.createElement("div");
```

```
card.className = "card";
```

```
const name = document.createElement("h2");
```

```
name.textContent = hero.Nombre;
```

- Aqui criamos um <div> para o card e um <h2> para o nome do personagem.
- Esses elementos não existiam no HTML original; foram gerados via JavaScript.

4.2.4. *appendChild()*

- **O que é:** função do DOM usada para inserir um elemento dentro de outro já existente.
- **No nosso código:**

javascript

```
card.appendChild(name);
```

```
card.appendChild(img);
```

```
card.appendChild(ocupacion);
```

```
card.appendChild(estado);
```

```
container.appendChild(card);
```

- Cada parte do card (nome, imagem, ocupação, estado) é adicionada ao <div class="card">.
- Depois, o card completo é inserido no container principal da página (#cards-container).

5. Integração entre as funções

O fluxo completo combina todas essas funções:

1. `fetch()` → requisita dados da API.
2. `await` → espera a resposta e converte para JSON.
3. `createElement()` → cria elementos HTML para cada personagem.
4. `appendChild()` → insere os elementos na página.

Esse conjunto transforma **dados brutos da API** em **conteúdo visual interativo** que o usuário vê em tempo real.

Conclusão

Essas funções são a base da aplicação:

- `fetch` e `await` cuidam da **comunicação com o servidor**.
- `createElement` e `appendChild` cuidam da **manipulação do DOM**.
- Juntas, elas permitem que o projeto consuma dados reais e os apresente de forma dinâmica e responsiva, sem recarregar a página.

5.1 Tratamento de erros e robustez da aplicação

Um aspecto importante do projeto é prever situações em que a API não esteja disponível ou o usuário esteja sem conexão com a internet. Para isso, utilizamos o bloco `try...catch` em JavaScript.

- **try**: executa a requisição normalmente.
- **catch**: captura qualquer erro (como falha de rede ou servidor fora do ar) e exibe uma mensagem amigável ao usuário.

Exemplo no código:

```
javascript
```

```
try{  
    const heroes = await searchHero(name);  
    // processamento normal  
} catch (error) {  
    container.textContent = " ⚠ Não foi possível conectar à API. Tente novamente mais tarde.";   
    container.className = "error-message";  
}
```

Esse tratamento garante que a aplicação não quebre em caso de falha e que o usuário receba feedback imediato, aumentando a confiabilidade e a usabilidade do sistema.

6. Motivo da escolha da API dos Simpsons (detalhado)

6.1. Relevância cultural

- **Os Simpsons** são uma das séries animadas mais icônicas dos anos 90 e 2000, com enorme impacto cultural e social.
- A série é reconhecida mundialmente, o que torna o projeto mais atrativo e fácil de contextualizar para qualquer público.
- A escolha reforça a proposta do trabalho, que pede uma API de desenhos de sucesso dessa época.

6.2. Facilidade técnica

- A API dos Simpsons é **pública e gratuita**, não exige cadastro nem autenticação por token.
- Isso simplifica a integração, pois basta usar o endpoint principal:

javascript

```
const BASE_URL = "https://apisimpsons.fly.dev/api/personajes?limit=1000";
```

- O limite de requisição (limit=1000) já retorna uma grande quantidade de personagens em uma única chamada, facilitando testes e exibição.

6.3. Estrutura dos dados

- A API retorna os dados em formato **JSON**, o que é ideal para consumo via JavaScript.
- Cada personagem possui atributos relevantes para a criação dos cards:
 - Nombre → Nome do personagem
 - Imagen → URL da imagem
 - Ocupacion → Ocupação ou profissão
 - Estado → Estado atual (vivo, morto, etc.)

Exemplo de resposta da API:

json

```
{
  "docs": [
    {
      "Nombre": "Homer Simpson",
      "Imagen": "https://...",
      "Ocupacion": "Operador na Usina Nuclear",
```

```
    "Estado": "Vivo"
  }
]
}
```

6.4. Aplicação prática no projeto

- Esses atributos permitem criar **cards ricos em informação** e visualmente atrativos.
- No nosso código, usamos:
 - `fetch()` para buscar os dados.
 - `filter()` para selecionar personagens pelo nome.
 - `createElement()` e `appendChild()` para montar os cards dinamicamente.

Exemplo:

javascript

```
const heroes = await searchHero("Homer");
heroes.forEach((hero) => {
  const card = createHeroCard(hero);
  container.appendChild(card);
});
```

6.5. Justificativa final

A escolha da API dos Simpsons une:

- **Cultura popular** (série famosa e reconhecida).
- **Facilidade técnica** (API aberta, sem autenticação).
- **Dados completos** (nome, imagem, ocupação, estado).
- **Adequação ao desafio** (desenho de sucesso dos anos 90/2000).

Isso garante que o projeto seja ao mesmo tempo **interessante para o usuário** e **viável tecnicamente** para o desenvolvedor.

7. Regras de acesso e documentação da API

- **Endpoint principal:** <https://apisimpsons.fly.dev/api/personajes?limit=1000>

- **Formato da resposta:** JSON, com o campo docs contendo um array de personagens.
- **Campos disponíveis:** Nombre, Imagen, Ocupacion, Estado.
- **Limite de requisição:** até 1000 personagens por chamada.
- **Autenticação:** não é necessária, a API é aberta.
- **Documentação:** disponível no próprio endpoint e em exemplos de uso com fetch.

Conclusão Geral

O desenvolvimento deste projeto permitiu compreender, de forma prática e aplicada, como o consumo de dados em tempo real aliado à manipulação dinâmica do DOM pode transformar informações brutas em interfaces visuais interativas. A utilização da API dos Simpsons demonstrou não apenas a relevância cultural de uma série icônica, mas também a viabilidade técnica de integrar uma fonte pública e gratuita de dados em uma aplicação web moderna.

Ao longo do trabalho, foram explorados conceitos fundamentais como requisições cliente-servidor, tratamento de respostas em JSON, criação e inserção de elementos HTML via JavaScript, além da importância de funções básicas como fetch, await, createElement e appendChild. Esses recursos, quando combinados, evidenciam a capacidade do front-end de oferecer experiências dinâmicas e responsivas ao usuário, sem a necessidade de recarregar a página.

Além disso, a implementação de mecanismos de tratamento de erros reforça a robustez da aplicação, garantindo que o usuário receba feedback adequado mesmo em situações de falha de conexão ou indisponibilidade da API. Esse cuidado evidencia a preocupação com a usabilidade e a confiabilidade do sistema.